From Solver Prototype to Maintainable Flight-Stack Integration: A Reproducible TinyMPC–PX4 Case Study

Ishaan Mahajan

Abstract— Research controllers are often released as solver examples or isolated simulations, while integration with an actively maintained robot stack depends on generated glue, implicit coordinate conventions, and manually reproduced demonstrations. We report an ongoing case study integrating TinyMPC with PX4 through a narrow native C++ boundary. TinyMPC consumes PX4 state estimates and publishes acceleration and yaw-rate commands at 50 Hz; PX4 retains estimation, attitude and rate stabilization, control allocation, arming, logging, and failsafe handling. The same controller implementation is exercised by deterministic native tests and by PX4/Gazebo Software-in-the-Loop (SITL) experiments. Versioned scenarios, explicit constraint semantics, solver diagnostics, residual-gated solution acceptance, and fail-closed transitions make experimental behavior inspectable rather than hiding it behind a successful video. A matched no-wind chicane illustrates the workflow: TinyMPC remained within a 0.36 m position corridor, while PX4's stock cascaded outer controller departed by 0.249 m under the same reference and inner flight stack. We use the case study to identify practices for sustaining optimization-based robotics software: stable control boundaries, one-source test ladders, machine-readable evidence, and clear separation of supported, legacy, and experimental configurations.

1. Motivation

TinyMPC provides an efficient ADMM-based model-predictive control solver for resource-constrained robots [1], including generated support for second-order-cone constraints [2]. PX4 provides a mature publish–subscribe flight stack spanning estimation, control, allocation, logging, and safety services [3]. Connecting the two is superficially a matter of publishing a setpoint. In practice, a credible integration must also define who owns each feedback loop, how solver failures are handled, which coordinate and actuator conventions are assumed, and how a result can be reproduced after either dependency changes.

Our repository initially accumulated several valid but overlapping paths: Simulink-generated guidance control, generated direct-acceleration control, native deterministic tests, and an experimental motor-level controller. Each answered a different research question, but their coexistence made it difficult for a new user to determine which configuration was the maintained integration. This experience motivates the narrower architecture and explicit maturity labels described here.

2. A narrow native control boundary

The maintained research path replaces only PX4's outer position/velocity feedback. At each 20 ms update, TinyMPC reads local position, velocity, and yaw, solves a 25-knot translational MPC problem, and publishes acceleration and yaw-rate. PX4 converts that acceleration to attitude and collective thrust, closes the faster attitude/rate loops, and allocates motor commands. Position and velocity fields are intentionally unset, preventing PX4 from silently adding a second outer feedback loop.

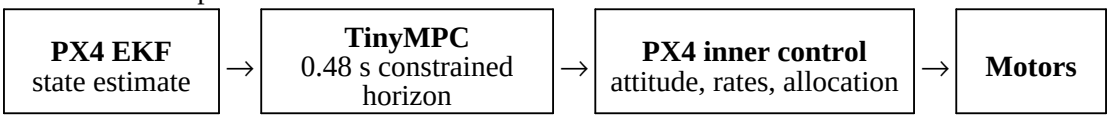

Fig. 1. The native acceleration-level boundary. PX4 continues to own estimation and safety-critical inner flight-stack services.

This boundary is deliberately less ambitious than direct motor control. It lets the predictive controller express future position, velocity, acceleration, and coupled tilt/thrust constraints while reusing PX4's tested infrastructure. A separate torque/thrust-level module remains explicitly experimental and SITL-only; it is not part of the supported path or the evaluation below.

3. Reproducibility and failure policy

The native benchmark and PX4 module compile the same TinyMPC wrapper and scenario definitions. Tests therefore exercise the deployed optimization code rather than a reimplementation in a plotting script. The workflow has three layers: deterministic ideal-model assertions, PX4 firmware compilation against a pinned PX4 release, and closed-loop Gazebo runs evaluated from PX4 ULogs. Scenario geometry and telemetry rendering are stored as code, and quantitative limits are checked before visual artifacts are produced [4].

Solver status is treated as part of the controller API. A converged finite solution is accepted; a fixed-budget solution is accepted only after residual and projected-constraint checks; otherwise the controller uses a bounded hover-oriented fallback. The PX4 adapter additionally checks estimator freshness and resets, finite commands, state-envelope departure, and an execution deadline. A runtime guard requests autonomous Loiter instead of continuing to advertise a valid Offboard controller. These policies do not prove physical safety, but they make failure behavior deterministic and testable.

4. Matched no-disturbance case study

We evaluated two outer controllers on the same PX4 v1.15.3 Gazebo X500, EKF, attitude/rate loops, allocation, and 15-degree tilt setting. Both received a 4.4 s reference containing two alternating 90-degree turns through the union of three 0.36 m-wide rectangles. There was no wind, collision, or injected disturbance. TinyMPC optimized the complete 0.48 s preview with corridor, velocity, altitude, input, and coupled tilt/thrust constraints. The baseline used PX4's stock cascaded position/velocity controller with the same current position and velocity reference.

Evaluation and controller	Maximum outside corridor	Final error / completion
Deterministic: TinyMPC	0.000 m	0.0025 m; 0 fallbacks
Deterministic: PX4 equations	0.231 m	0.0106 m
PX4/Gazebo: TinyMPC → PX4	0.000 m	1,095 solves; no module failure
PX4/Gazebo: stock PX4 outer loop	0.249 m	Completed and landed normally

Table 1. Matched deterministic and SITL evidence; replay and artifact: [4].

The successful TinyMPC SITL run observed a 1.068 ms worst host solve, but this is not Pixhawk timing evidence. The result also does not establish that stock PX4 is generally unable to traverse the course: a slower, wider, or pre-smoothed reference could make the task easy for cascaded control. The demonstrated architectural difference is that TinyMPC includes the future spatial envelope and coupled input limit in one optimization, whereas the stock outer loop reacts to the current reference and relies on downstream saturation.

5. Maintenance lessons

- **Name one supported boundary.** "TinyMPC–PX4" is insufficient when the repository contains guidance-, acceleration-, and actuator-level integrations. We now designate native acceleration control as the supported research path, stock PX4 as the baseline, motor-level control as experimental, generated Simulink integrations as legacy, and native simulations as tests.
- **Test the deployed implementation.** Sharing the controller wrapper, constraints, and diagnostics between native regression and PX4 avoids agreement by transcription. Every scenario must define its origin, units, horizon, constraint semantics, fallback, and measurable acceptance criteria.
- **Preserve the host stack's responsibilities.** A narrow setpoint interface reduces the code that must be maintained across PX4 releases and leaves estimation, allocation, logging, and mode management with their established owners.
- **Publish negative scope.** SITL timing is not embedded timing; predicted constraints are not guaranteed physical envelopes; and successful integration is not hardware validation. Recording these limits prevents an experimental branch from becoming an undocumented deployment path.

6. Limitations and next steps

The native module currently refuses non-POSIX builds. Before hardware control, it requires a PX4/NuttX port, target measurements of solve latency and memory, a smaller target-derived iteration budget, estimator-discontinuity tests, and conservative tethered validation. The current 100-iteration maximum and 64 kB task stack are research settings, not Pixhawk-qualified choices. Continuous integration presently covers native compilation and deterministic controller assertions; automating PX4 firmware compilation, Gazebo log acceptance, and machine-readable run manifests are the next sustainability steps.

7. Conclusion

The principal contribution of this case study is not a new flight stack, but a maintainable seam between an optimization library and one. Explicit ownership, shared tests, inspectable failure policies, and honest maturity labels turn a one-off controller demonstration into a reproducible research artifact. The same practices apply broadly when learning- or model-based controllers are integrated with long-lived robotics infrastructure.

References

[1] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "TinyMPC: Model-Predictive Control on Resource-Constrained Microcontrollers," *Proc. IEEE ICRA*, 2024.

[2] I. Mahajan, K. Nguyen, S. Schoedel, E. Nedumaran, M. Mata, B. Plancher, and Z. Manchester, "Code Generation and Conic Constraints for Model-Predictive Control on Microcontrollers with Conic-TinyMPC," *Proc. IEEE ICRA*, 2026.

[3] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms," *Proc. IEEE ICRA*, pp. 6235–6240, 2015. doi:10.1109/ICRA.2015.7140074.

[4] "TinyMPC–PX4 chicane artifact and replay," commit `6abd74b`, 2026. github.com/TinyMPC/tinympc-px4/tree/main.